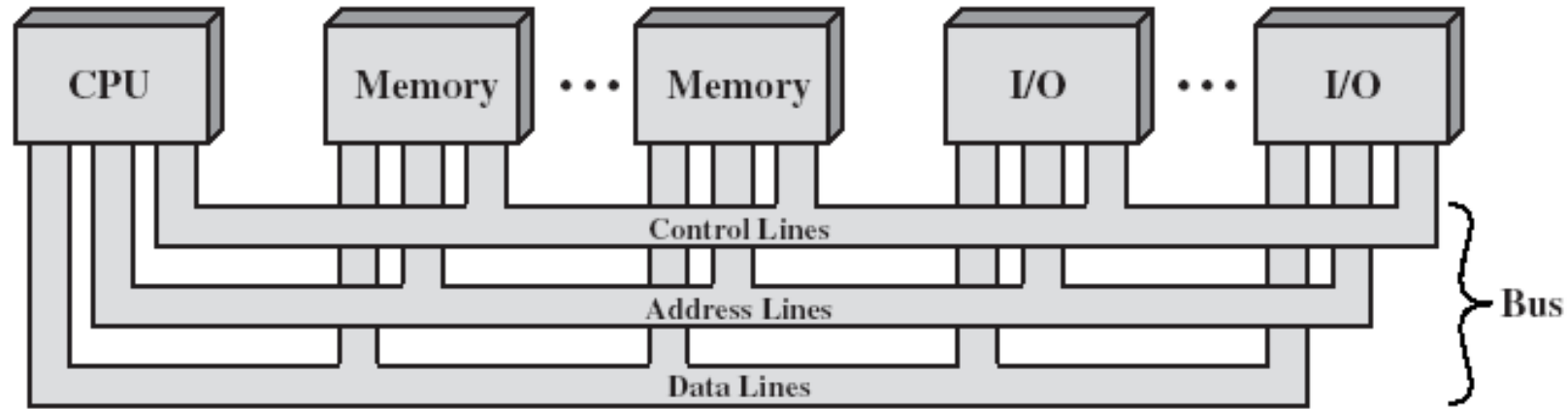


Architettura degli Elaboratori

Il Bus

Bus - Architettura Classica



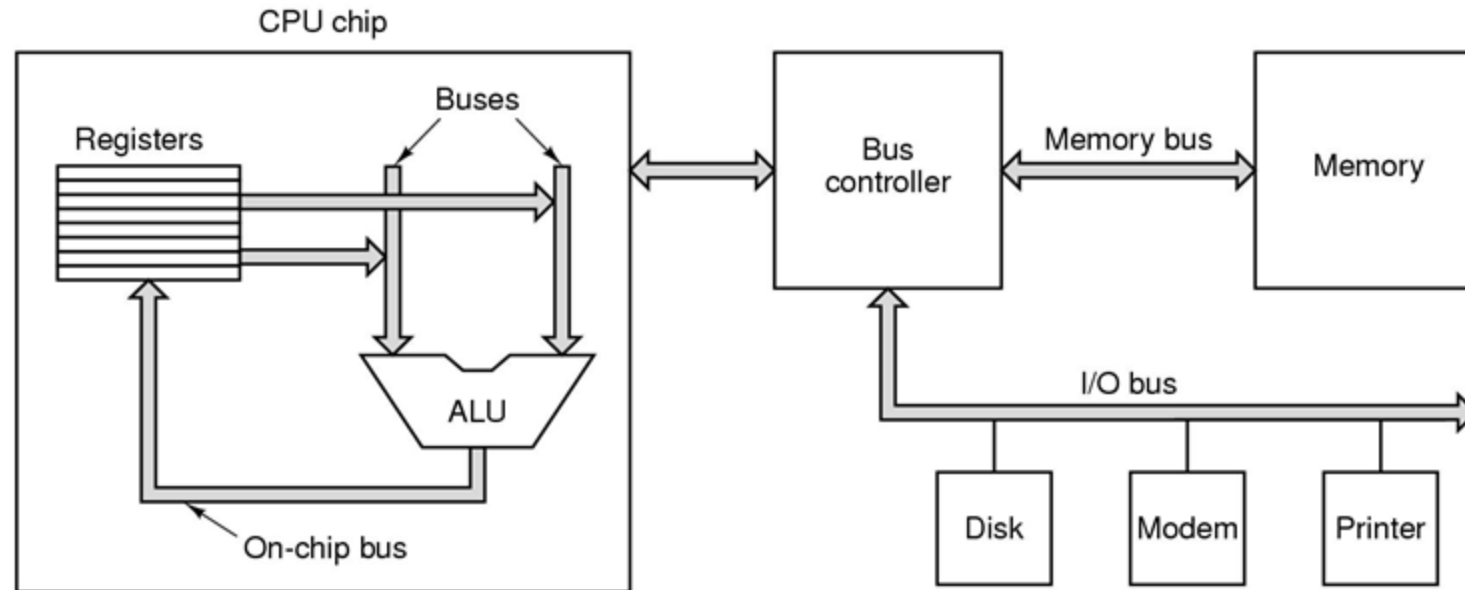
Sistemi tradizionali (figura)

- . Bus condiviso
- . Linee di controllo
- . Linee dati
- . Linee indirizzi

Sistemi moderni

- . Connessioni punto-punto
- . Controller dedicati
(es. memory controller, I/O controller)

Bus - Architettura Classica



- Il bus è l'insieme di linee elettriche che collegano i moduli di un elaboratore
- Vari tipi di bus (con dimensioni fisiche molto diverse):
 - **Bus di sistema**: interconnette la CPU, le schede di I/O e la memoria
 - **Bus interni al chip**: mettono in comunicazione i moduli della CPU
 - ...
- L'insieme delle regole viene definito come il **protocollo del bus**

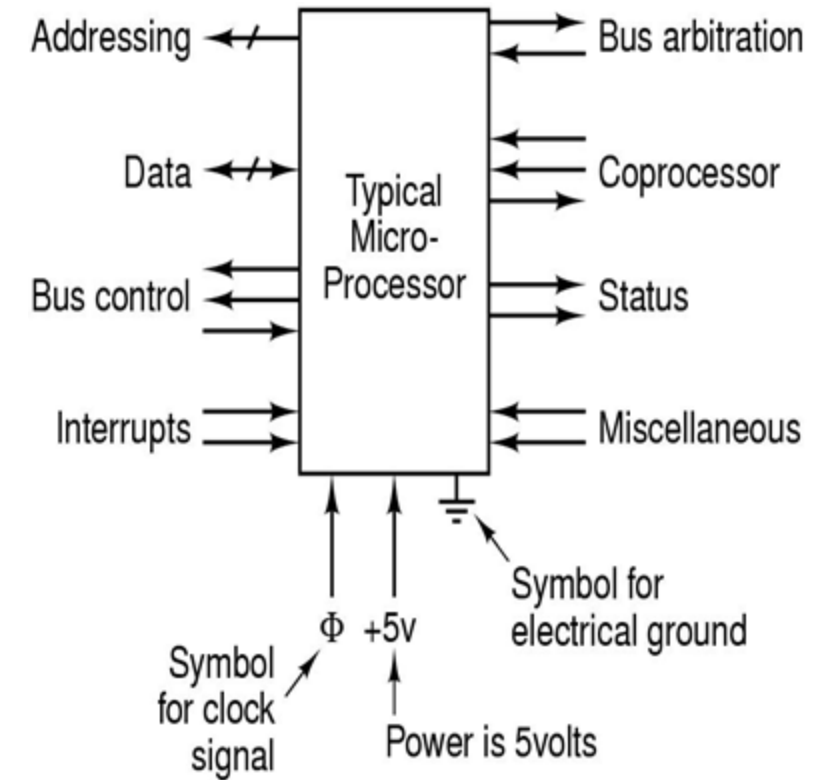
Bus - Architettura Classica

Le linee del bus possono essere linee di dati, di indirizzo e di controllo

- **Linee di dati:** Il numero di linee (**larghezza** del data bus) determina il numero di bit che possono essere trasmessi alla volta: ha un impatto sulle prestazioni del sistema.
- **Linee di indirizzo:** permettono di individuare la sorgente/destinazione dei dati trasmessi sul data bus.
- **Linee di controllo:** controllano l'accesso e l'utilizzo delle linee di dati e di indirizzo

Connessioni di una CPU

- **Indirizzo:** n piedini corrispondono a 2^n locazioni di memoria indirizzabili (valori tipici: $n = 16, 20, 32, 64$)
- **Dati:** n piedini permettono di leggere/scrivere una parola di n bit con una sola operazione (valori tipici $n = 8, 16, 32, 36, 64$)
- **Controllo:** regolano il flusso e la scansione dei dati verso e dal chip. Categorie principali:
 - controllo del bus
 - interrupt
 - arbitraggio del bus
 - segnali del coprocessore
 - stato
 - varie



Dispositivi attivi e passivi

I dispositivi collegati ad un bus si dividono in

- **attivi**: possono decidere di iniziare un trasferimento, in genere sono collegati al bus per mezzo di un particolare chip detto *bus driver*
- **passivi**: rimangono in attesa di richieste, in genere sono collegati per mezzo di un chip detto *bus receiver*
- I dispositivi che si comportano sia come attivo che come passivo (es. la CPU) sono collegati attraverso un chip combinato: il *bus transceiver*

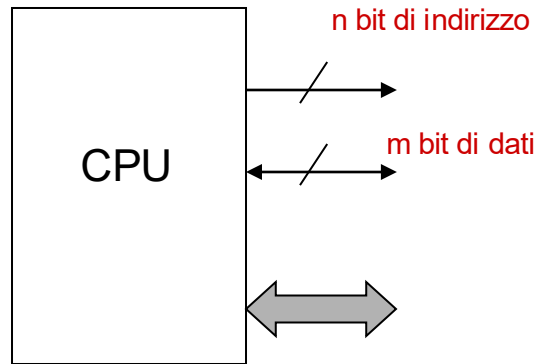
Master	Slave	Example
CPU	Memory	Fetching instructions and data
CPU	I/O device	Initiating data transfer
CPU	Coprocessor	CPU handing instruction off to coprocessor
I/O	Memory	DMA (Direct Memory Access)
Coprocessor	CPU	Coprocessor fetching operands from CPU

Progettazione di un bus

I problemi principali nella progettazione di un bus riguardano

- **Larghezza** del bus: (numero di linee)
- **Arbitraggio**: come scegliere tra due dispositivi che vogliono diventare contemporaneamente arbitri dello stesso bus
- **Funzionamento** del bus: come avviene il trasferimento dei bit

Larghezza del bus



È possibile indirizzare 2^n celle

e trasferire m bit alla volta

- Il numero delle linee utilizzate per trasferire gli indirizzi determinano la massima quantità della memoria indirizzabile
- Il numero delle linee utilizzate per il trasferimento di dati determinano la quantità di informazioni che è possibile trasferire con una singola operazione

Bus clocking

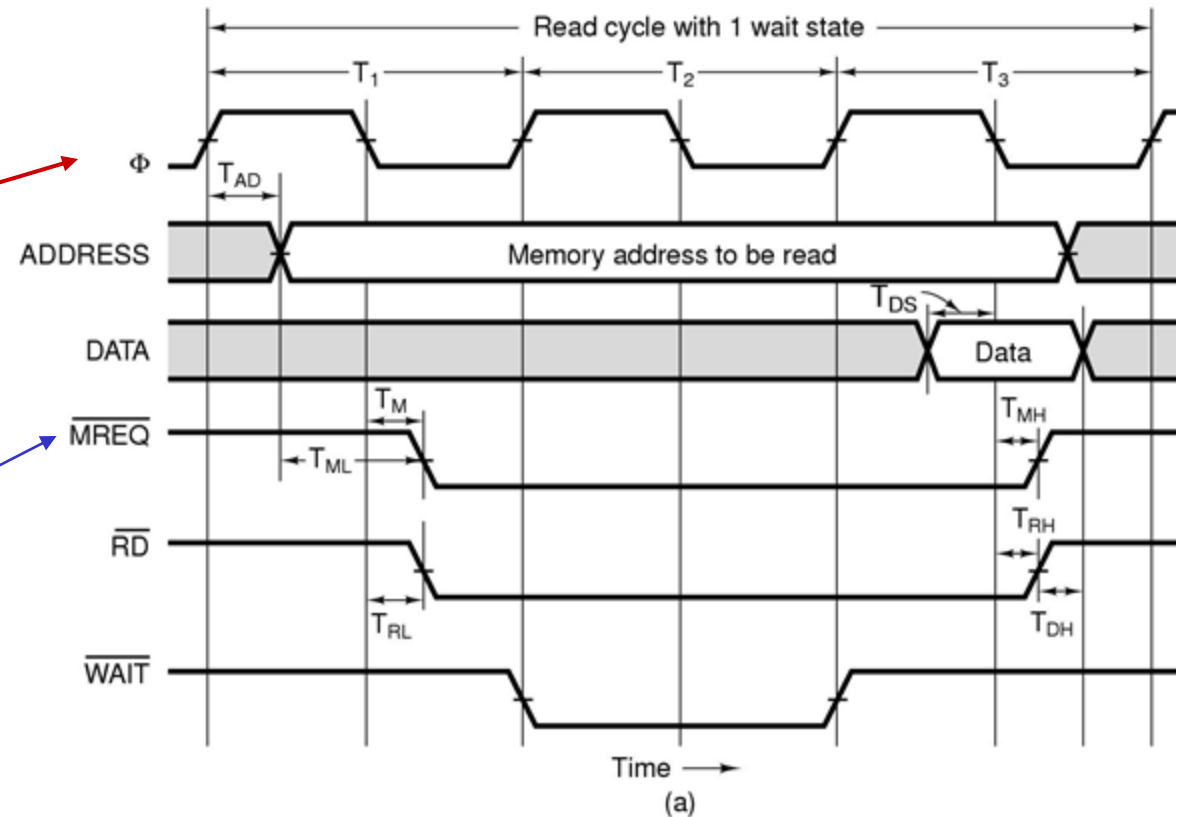
I bus si possono dividere in due categorie:

- **Bus sincroni:** usano una linea di clock comune (pilotata da un oscillatore), con tutte le attività che avvengono in multipli interi del ciclo di clock. Esempi classici includono ISA, PCI e DDR. Le frequenze possono variare da pochi MHz (ISA) a diversi GHz (DDR5, PCIe).
- **Bus asincroni:** non usano un clock globale; ogni transazione può richiedere tempi diversi, sincronizzandosi tramite segnali di handshaking.

Nota: Nei sistemi moderni, molte interconnessioni combinano elementi sincroni e asincroni, e spesso sono serializzate.

Bus sincrono

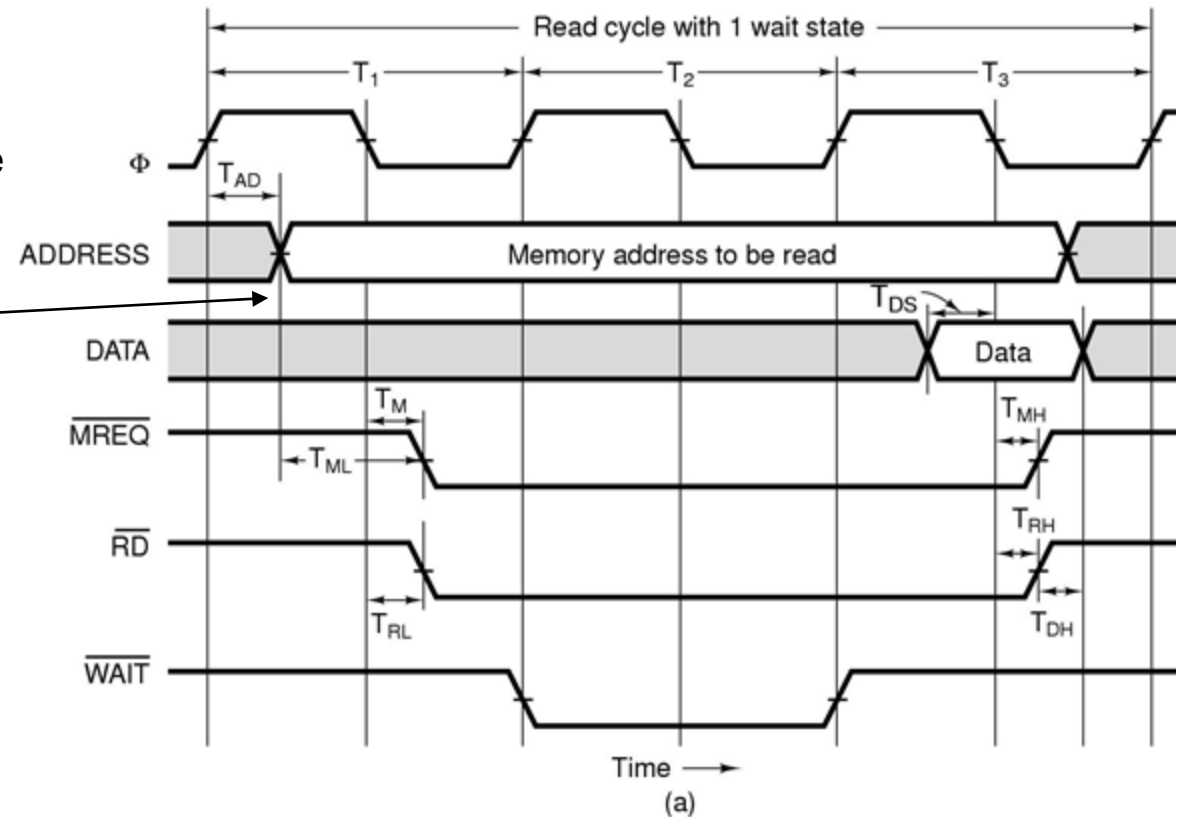
- Il bus sincrono ha una linea pilotata da un **oscillatore**; ogni attività richiede un numero intero di cicli.
- La durata delle fasi è nota ad entrambi i partecipanti e l'unica incognita è l'inizio della comunicazione.
- La linea **MREQ** indica l'inizio della comunicazione



Bus sincrono – Ciclo di lettura

Supponiamo di avere un clock di 100 Mhz
(ciclo di 10 nsec.) e che la lettura dalla
memoria richieda 15 nsec da quando l'indirizzo è stabile

La **CPU** mette l'indirizzo sul bus, dopo un certo
tempo necessario per stabilizzarsi (T_{AD}),
l'indirizzo rimane disponibile

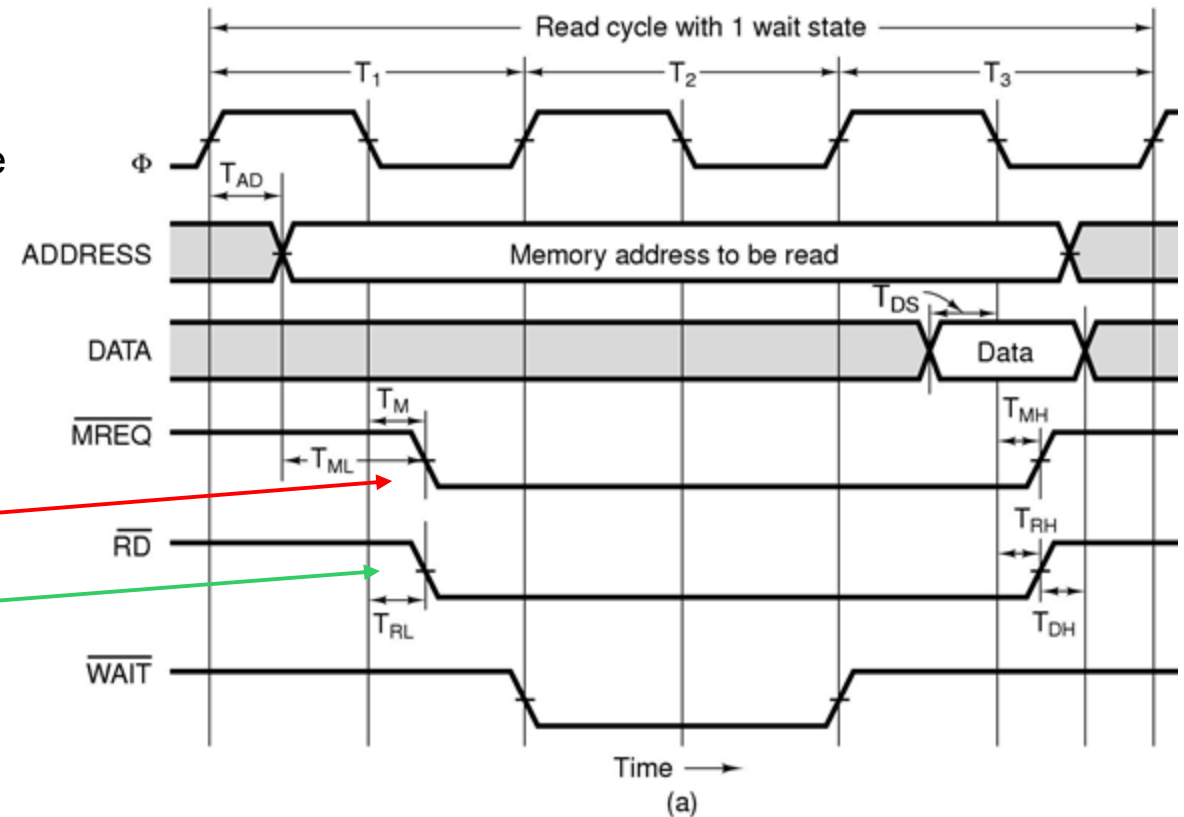


Bus sincrono – Ciclo di lettura

Supponiamo di avere un clock di 100 Mhz (ciclo di 10 nsec.) e che la lettura dalla memoria richieda 15 nsec da quando l'indirizzo è stabile

La **CPU** mette l'indirizzo sul bus, dopo un certo tempo necessario per stabilizzarsi (T_{AD}), l'indirizzo rimane disponibile

A linee di indirizzo stabili la **CPU** attiva i segnali di richiesta di **inizio comunicazione** e il **segnale di lettura**



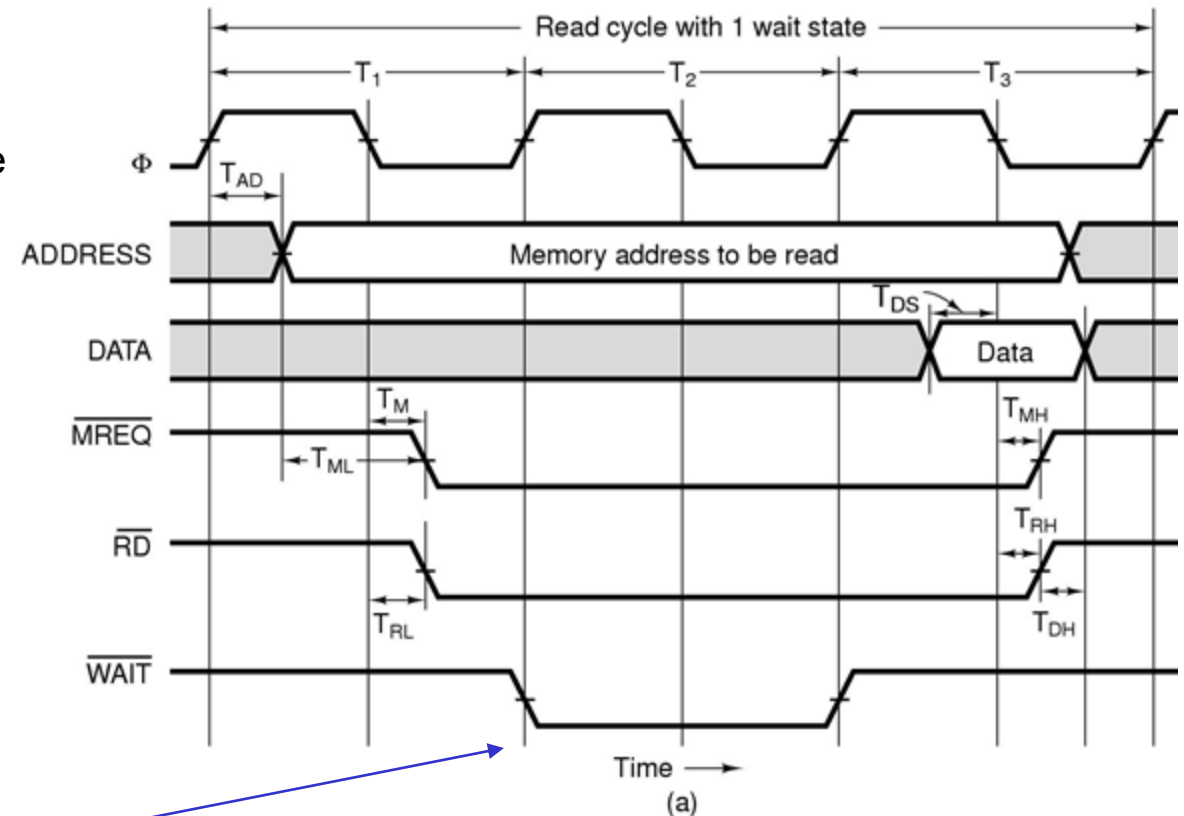
Bus sincrono – Ciclo di lettura

Supponiamo di avere un clock di 100 Mhz (ciclo di 10 nsec.) e che la lettura dalla memoria richieda 15 nsec da quando l'indirizzo è stabile

La **CPU** mette l'indirizzo sul bus, dopo un certo tempo necessario per stabilizzarsi (T_{AD}), l'indirizzo rimane disponibile

A linee di indirizzo stabili la **CPU** attiva i segnali di richiesta di **inizio comunicazione** e il **segnale di lettura**

Per avvertire la CPU di non aspettare i dati, all'inizio del ciclo T_2 , la **memoria** attiva il segnale di **attesa dati**



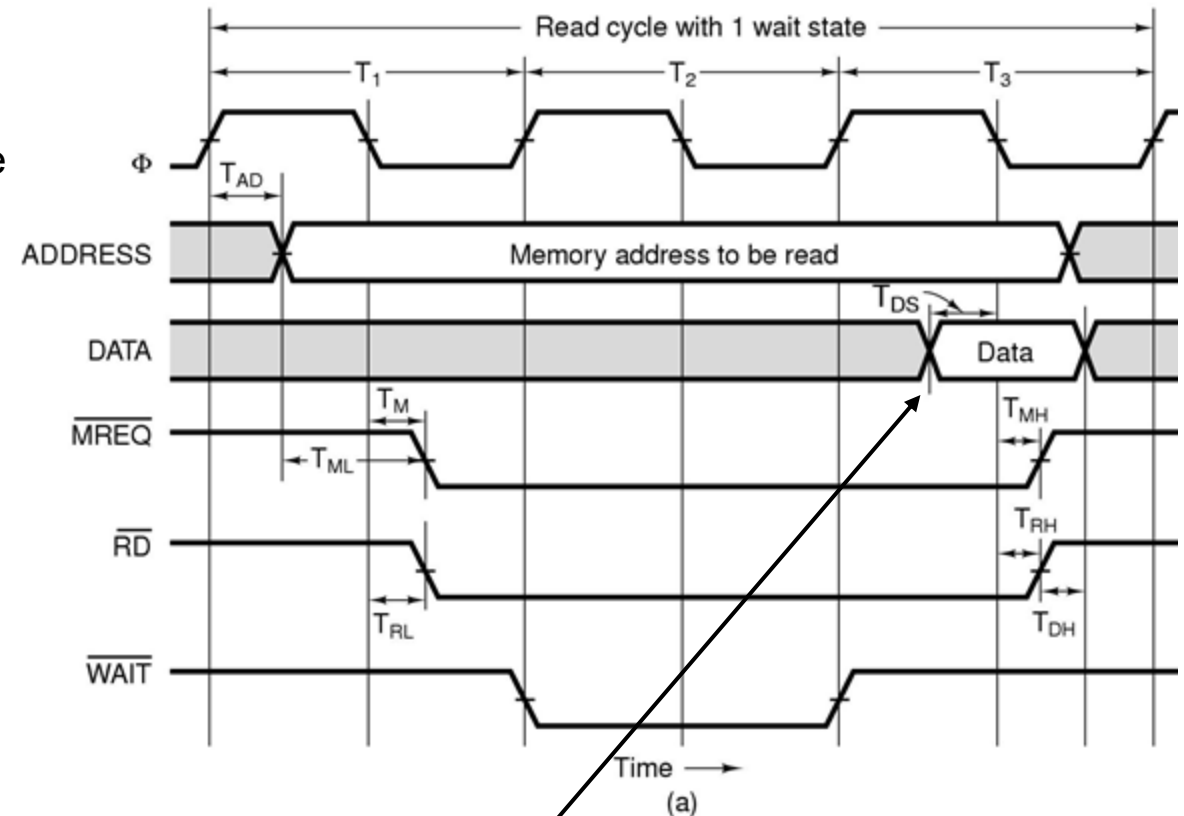
Bus sincrono – Ciclo di lettura

Supponiamo di avere un clock di 100 Mhz (ciclo di 10 nsec.) e che la lettura dalla memoria richieda 15 nsec da quando l'indirizzo è stabile

La **CPU** mette l'indirizzo sul bus, dopo un certo tempo necessario per stabilizzarsi (T_{AD}), l'indirizzo rimane disponibile

A linee di indirizzo stabili la **CPU** attiva i segnali di richiesta di **inizio comunicazione** e il **segnale di lettura**

Per avvertire la CPU di non aspettare i dati, all'inizio del ciclo T_2 , la **memoria** attiva il segnale di **attesa dati**



La **memoria** mette i dati sulla linea dati

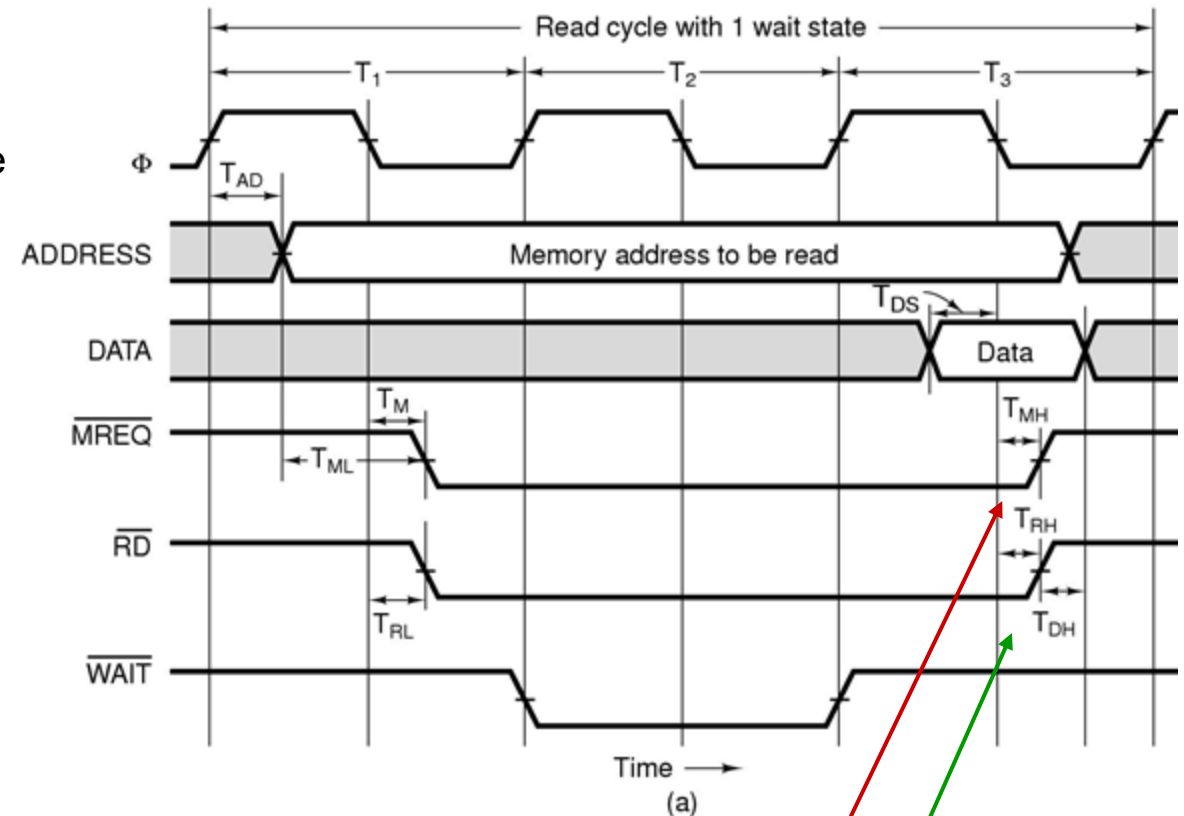
Bus sincrono – Ciclo di lettura

Supponiamo di avere un clock di 100 Mhz (ciclo di 10 nsec.) e che la lettura dalla memoria richieda 15 nsec da quando l'indirizzo è stabile

La **CPU** mette l'indirizzo sul bus, dopo un certo tempo necessario per stabilizzarsi (T_{AD}), l'indirizzo rimane disponibile

A linee di indirizzo stabili la **CPU** attiva i segnali di richiesta di **inizio comunicazione** e il **segnale di lettura**

Per avvertire la CPU di non aspettare i dati, all'inizio del ciclo T_2 , la **memoria** attiva il segnale di **attesa dati**

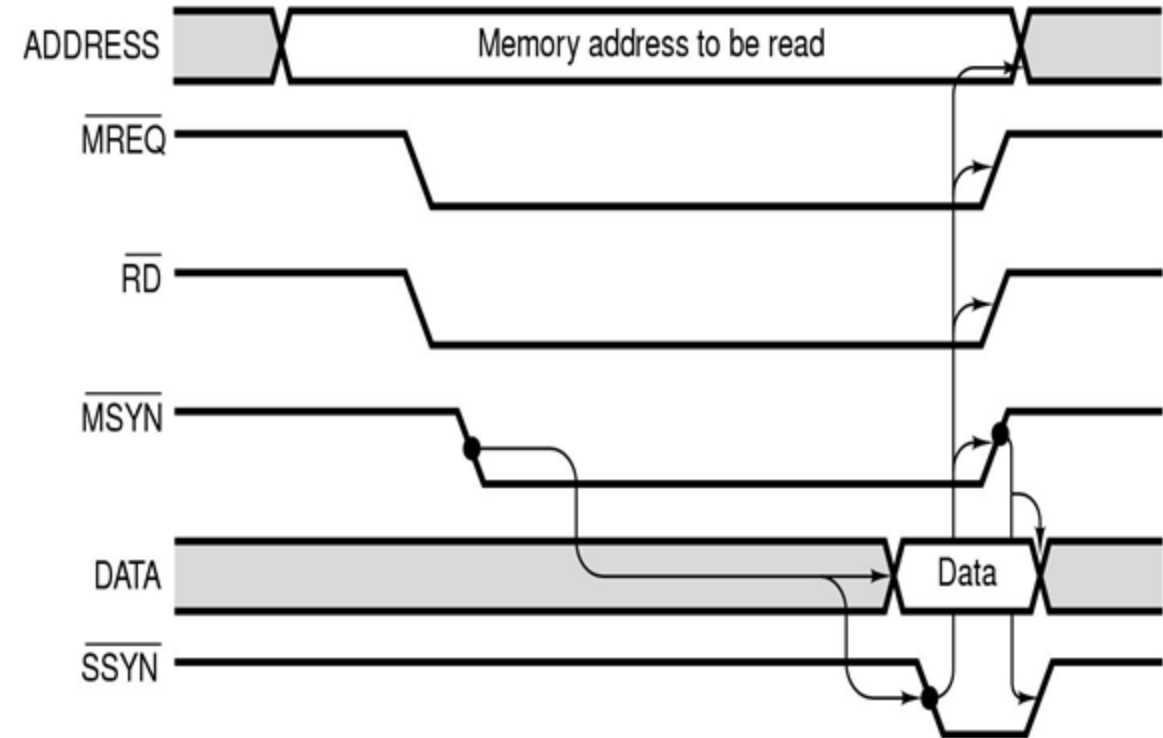


La **memoria** mette i dati sulla linea dati

Dopo aver letto i dati la **CPU** disattiva i segnali di **richiesta** e di **lettura**

Bus asincrono

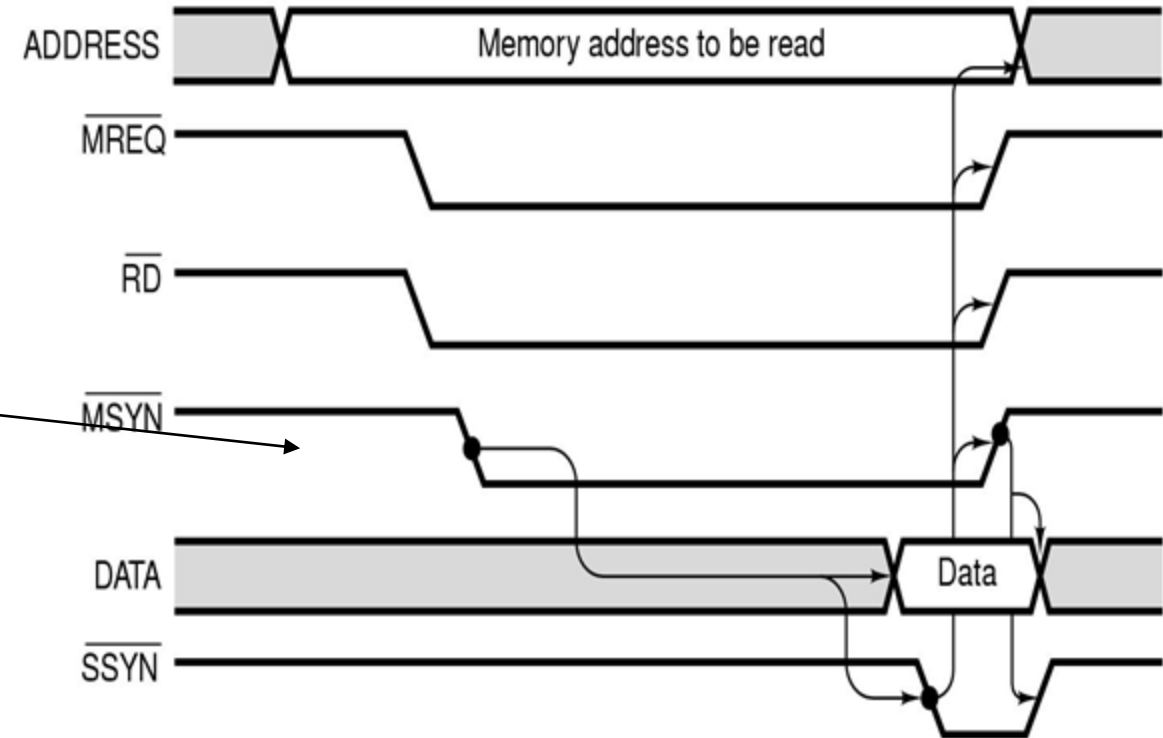
- Il bus sincrono deve adattarsi al dispositivo più lento
- In questi casi funziona meglio un **bus asincrono**, cioè senza un clock



Bus asincrono

- Il bus sincrono deve adattarsi al dispositivo più lento
- In questi casi funziona meglio un **bus asincrono**, cioè senza un clock

Attivati i segnali di accesso alla memoria e lettura, attiva il segnale SYNchronization

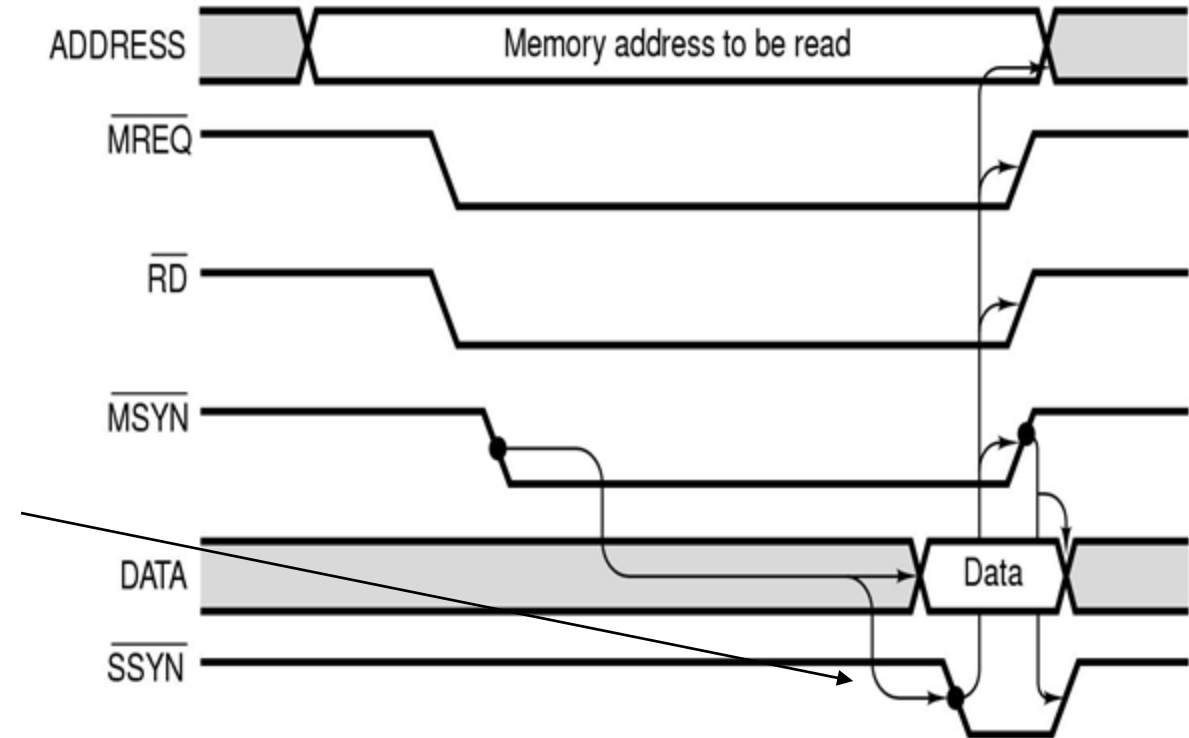


Bus asincrono

- Il bus sincrono deve adattarsi al dispositivo più lento
- In questi casi funziona meglio un **bus asincrono**, cioè senza un clock

Attivati i segnali di accesso alla memoria e lettura, attiva il segnale SYNchronization

Il device esegue il suo lavoro all'attivazione del segnale e quindi terminatolo attiva il segnale di SYNchronization



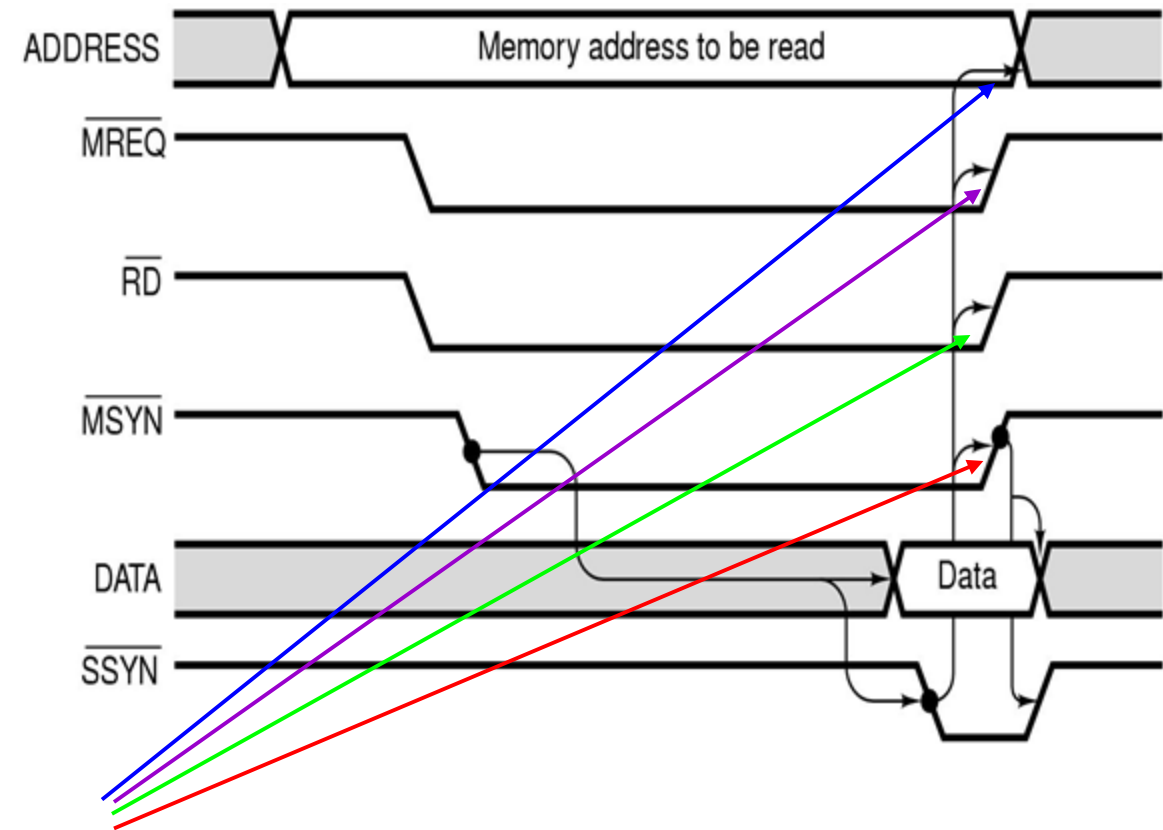
Bus asincrono

- Il bus sincrono deve adattarsi al dispositivo più lento
- In questi casi funziona meglio un **bus asincrono**, cioè senza un clock

Attivati i segnali di accesso alla memoria e lettura, attiva il segnale SYNchronization

Il device esegue il suo lavoro all'attivazione del segnale e quindi terminatolo attiva il segnale di SYNchronization

I **dati vengono memorizzati** dal device che nega i segnali di **accesso alla memoria**, **lettura** e **SYNchronization**



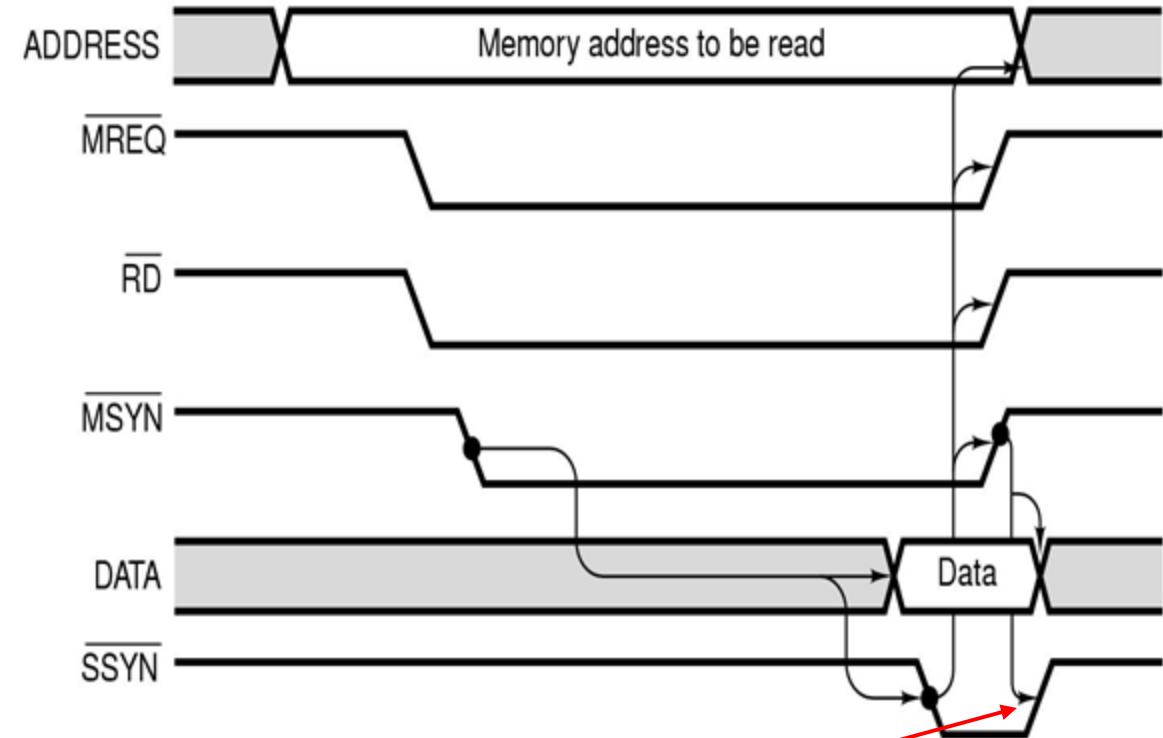
Bus asincrono

- Il bus sincrono deve adattarsi al dispositivo più lento
- In questi casi funziona meglio un **bus asincrono**, cioè senza un clock

Attivati i segnali di accesso alla memoria e lettura, attiva il segnale SYNchronization

Il device esegue il suo lavoro all'attivazione del segnale e quindi terminatolo attiva il segnale di SYNchronization

I **dati vengono memorizzati** dal device che nega i segnali di **accesso alla memoria**, **lettura** e **SYNchronization**



Il device dopo aver visto la negazione del segnale di synchronization, nega a sua volta il suo segnale di **SYNchronization**

Full handshake

Sincrono vs Asincrono

Bus sincrono

- **Vantaggi**

- Realizzazione device semplice
- Se la durata di un'operazione è fissa non occorre una linea di wait

- **Svantaggi**

- Durata di una operazione di comunicazione deve necessariamente avere una durata pari ad un numero intero di cicli

Bus asincrono

- **Vantaggi**

- Flessibilità: la durata di una operazione è determinata unicamente dalla velocità della coppia di device

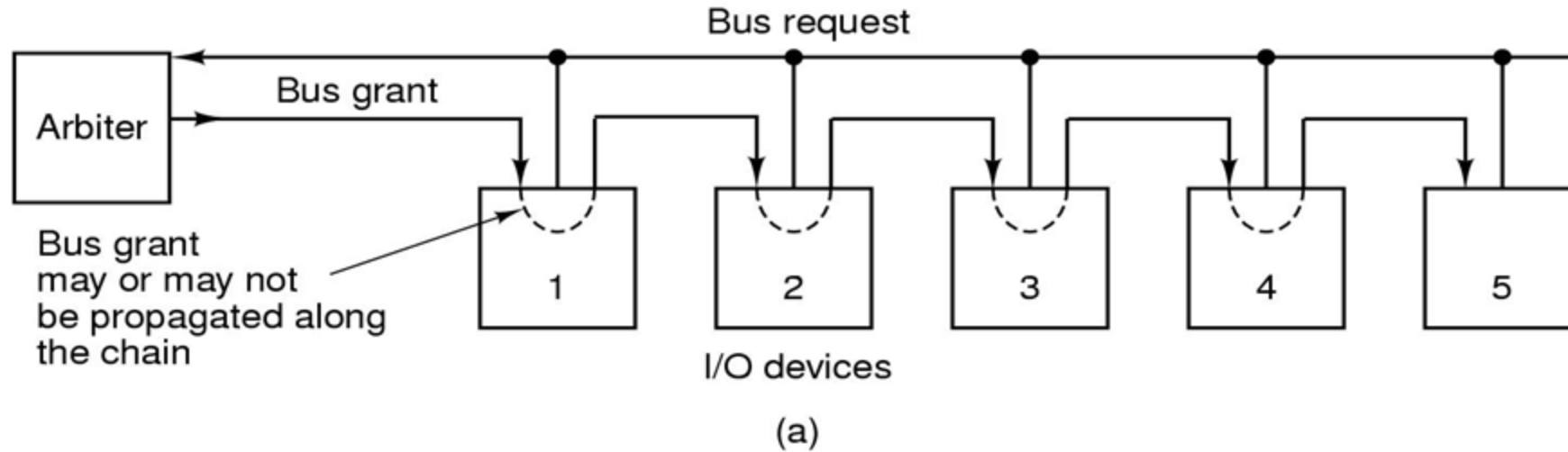
- **Svantaggi**

- Per completare una operazione di comunicazione sono sempre necessarie 4 azioni
- Occorre inserire nei device i circuiti necessari a rispondere opportunamente al protocollo

Arbitraggio del bus

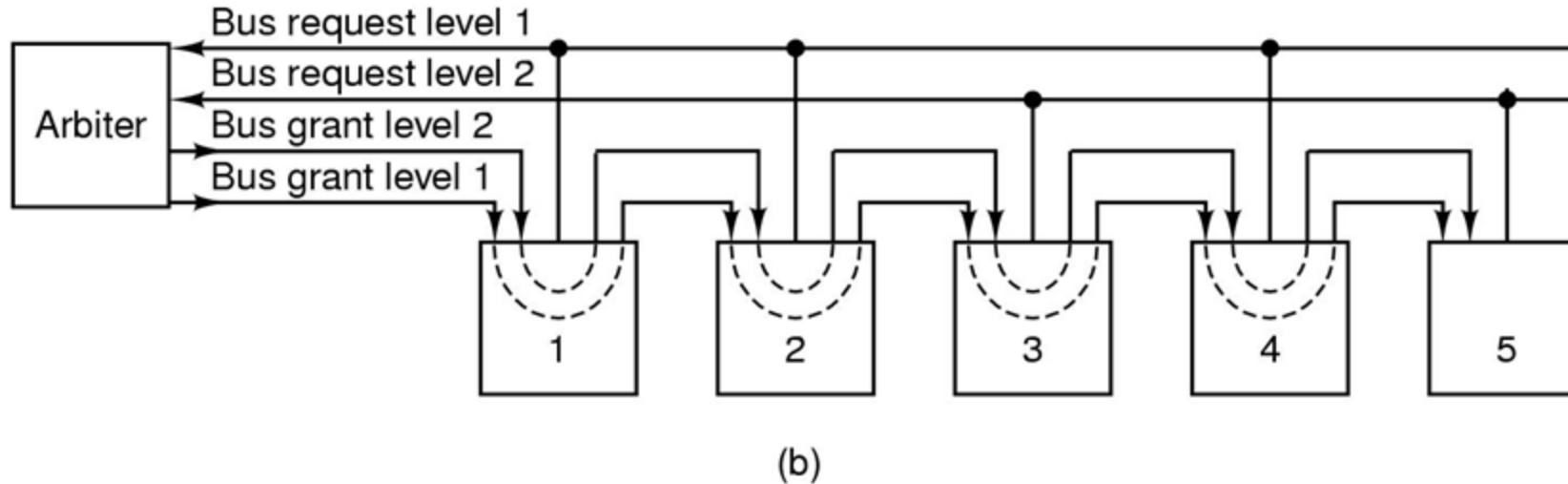
- Cosa succede se più di un dispositivo richiede l'utilizzo del bus contemporaneamente?
- È necessario un meccanismo di **arbitraggio del bus** per evitare ambiguità delle informazioni immesse sul bus stesso, sono possibili due strade:
 - arbitraggio centralizzato
 - arbitraggio decentralizzato
- Meccanismo del **grant**

Arbitraggio del bus



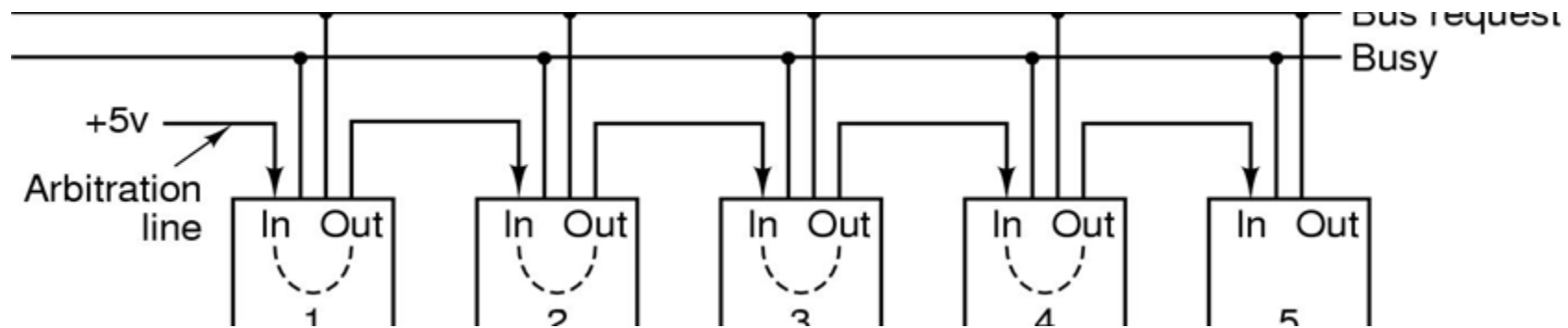
- **Arbitro centralizzato** quando l'arbitro "vede" una richiesta attiva la linea di *grant* del bus
- **Daisy chaining**: il *grant* viene trasmesso lungo la linea bus grant finché un dispositivo non accetta l'assegnamento
- Vince il dispositivo più vicino

Arbitraggio del bus



- Due (o più) livelli di priorità
- L'assegnamento segue lo stesso meccanismo del daisy chaining ma di dispositivi con priorità più alta hanno precedenza nell'assegnamento del grant da parte dell'arbitro
- Linea di **acknowledge**
- **Scelta attenta delle priorità tra i vari dispositivi**

Arbitraggio del bus

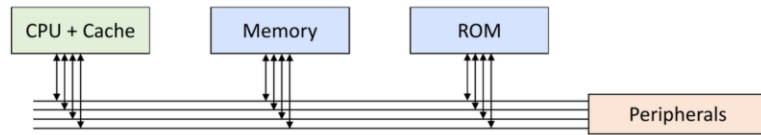


- Arbitro **decentralizzato**
 - Tante linee quanti dispositivi, ogni dispositivo osserva le linee prima di effettuare la richiesta (soluzione poco flessibile ma risparmia il costo dell'arbitro)
- Soluzione con **tre linee**, *bus request*, *busy* e *linea di arbitraggio*
 - Più economico e più veloce del daisy chaining centralizzato

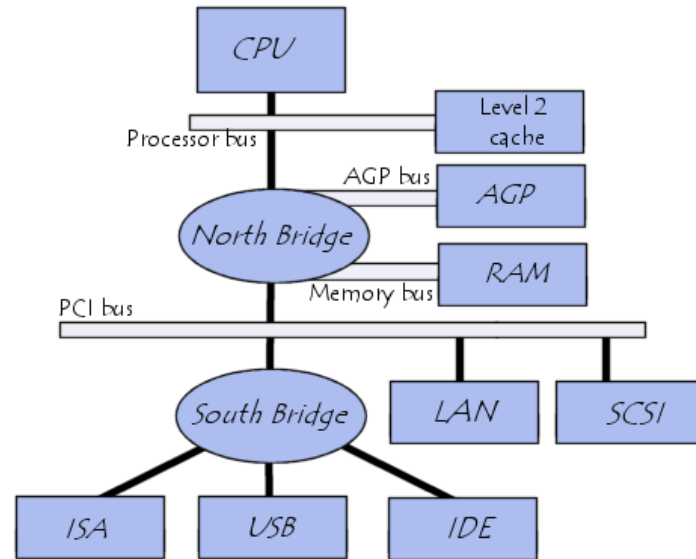
Da bus paralleli a seriali

- I bus tradizionali (ISA, EISA, PCI) utilizzano la trasmissione parallela: più linee trasmettono simultaneamente tutti i bit di una parola (es. 32 bit)
- La trasmissione parallela era considerata il metodo più veloce, ma presenta limiti pratici:
 - Interferenza tra linee (crosstalk), sincronizzazione, costi elevati, distanze limitate
- I bus seriali ad alta velocità (USB 3.x, PCI Express, Serial ATA, Thunderbolt) trasmettono i dati un bit alla volta, ma:
 - usano tecniche di codifica e serializzazione, raggiungono velocità molto superiori, minore interferenza, costi ridotti, distanze maggiori
- Esempi di bus seriali moderni:
 - PCI Express 6.0: fino a 64 GT/s per corsia
 - USB4 / Thunderbolt 4: fino a 40 Gbps
 - Serial ATA III: fino a 6 Gbps

Evoluzione storica



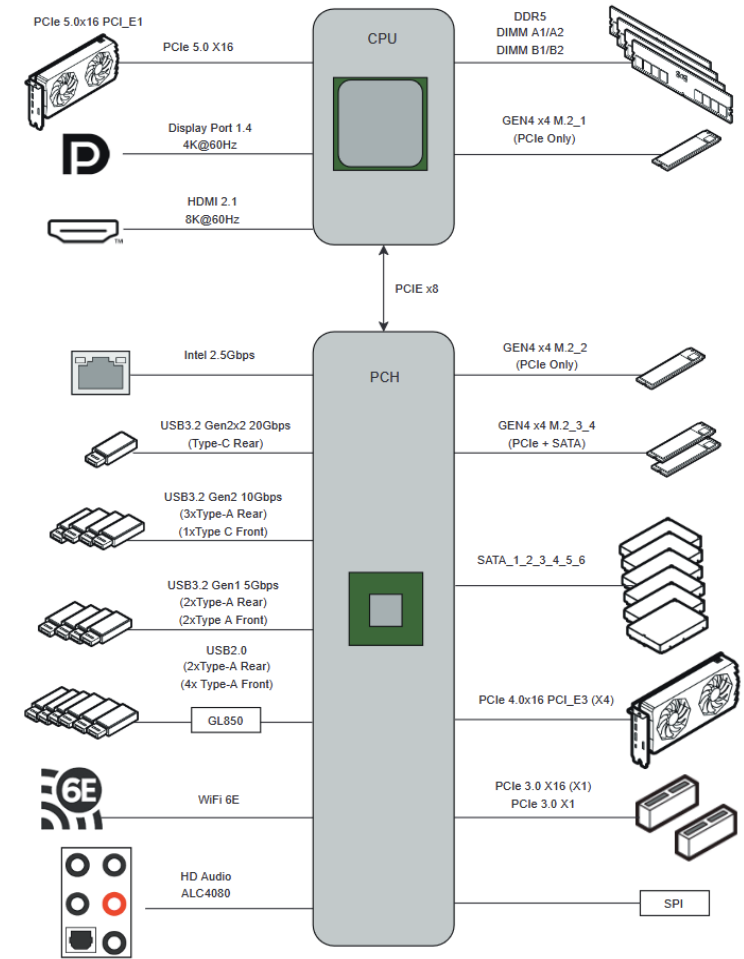
architettura classica



Northbridge/Southbridge
(fino agli anni 2010)

Sistemi attuali

- Bus seriale ad alta velocità (PCIe, USB, SATA, NVMe)
- Interconnessioni point-to-point (es. tra CPU e RAM, CPU e GPU)
- Sistemi SoC con interconnect avanzati (AMBA, AXI, NoC).



Intel Platform Controller Hub (PCH)
(fino agli anni 2020)